

Optimizing CNN Deployment on FPGA Using Hardware-Aware Neural Architecture Search

Amit Raj Pant ^{*1}, Pilot Khadka ^{*1}, Kshitiz Poudel ^{*1}, Bishal Rijal¹, Krishna Gaihre ^{†2}, and Shanta Maharjan ^{†1}

¹Department of Electronics and Computer Engineering, Thapathali Campus, IOE, Tribhuvan University, Kathmandu, Nepal

² LogicTronix Technologies Pvt. Ltd., Lalitpur, Nepal

Abstract—FPGAs offer incredible flexibility and cost efficiency in deploying real-time applications. However, not all Deep Neural Networks are optimized for FPGA deployment. Compression methods like quantization and pruning are often required, though they have drawbacks. This study revisits existing Neural Architecture Search (NAS) methodologies to discover efficient CNN architectures. We explore multiple approaches to NAS, targeting Ultra96V2 FPGA board and introduce a new constraint, the Compute Latency Index (CLI), and a weighted objective function to discover better-performing networks. We explore nearly 3×10^8 architectures within the MobileNetV3 search space, using the latency data derived from the FPGA to guide the evolutionary algorithm. Our results show that maximizing the arithmetic intensity results in deeper networks with higher latency, whereas latency-constrained search yields shallower networks. The CLI-guided search balances these conflicting objectives, discovering architectures that optimize accuracy, computational efficiency, and latency. We demonstrate that FPGA-specific parameter searches can yield Pareto-optimal architectures.

Index Terms—Deep Neural Networks, FPGAs, Multi-objective Optimization, Neural Architectural Search

I. INTRODUCTION

Deploying AI solutions at the edge where data is generated offers distinct advantages. Edge AI eliminates latency and security concerns associated with transmitting sensitive information to the cloud by processing the data locally. However, the problem of inferencing deep learning models in real-time requires substantial computation, primarily owing to the limited processing power and memory on edge devices.

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits that provide hardware-level programmability, enabling post-manufacturing customization of digital logic. Unlike conventional processors such as CPUs and GPUs, which rely on a predefined instruction set, FPGAs implement computations through spatial parallelism, mapping algorithms directly onto dedicated hardware circuits. This eliminates instruction-fetching overhead and allows for highly optimized data paths to specific workloads. Hardware description languages (HDLs) define the logic and interconnect structures within an FPGA. High-Level Synthesis (HLS) tools ease the FPGA programming by compiling high-level logic such as C, C++, and OpenCL into register-transfer level (RTL) representations, accelerating the development process. This

makes them an ideal choice for Edge AI because they reduce latency and allow real-time deployment of deep learning models on edge devices.

While FPGAs offer distinct advantages, deploying Convolutional Neural Networks (CNNs) on such platforms presents significant challenges. CNNs apply filters across input images to extract features like edges or complex patterns. CNNs have evolved to become deeper and more complicated to enhance feature extraction, learning capacity, and generalization. However, this complexity comes at a cost when considering their deployment on FPGAs. Their increasing memory and processing requirements exceed those of many FPGAs. One approach to addressing this challenge is to design an optimal model tailored to a specific FPGA and a particular problem domain. But this presents a significant challenge- it requires extensive experimentation, knowledge on network architectures and their limitations, and a careful consideration on an array of design options.

Recently, techniques such as Neural Architecture Search (NAS) and Hardware-aware NAS (HW-NAS) have emerged to address the complexity of designing optimal networks. These methods can automate the architecture search process, save time and effort, and allow for the exploration of complex architectures that human designers might overlook. However, traditional NAS techniques require large computational resources, and the search process can take days as every architecture needs to be evaluated to assess its performance. Instead, heuristic-based approach can be used by developing predictors that can predict the efficiency of the network based on its architecture. Additionally, HW-NAS requires adding hardware constraints to the search process. Developing an estimator that can estimate these constraints becomes crucial for reducing the search time. If the constraints are optimized properly, it can deliver architectures with the desired latency and energy efficiency while achieving similar accuracy.

An alternative approach for optimizing DNNs for FPGA deployment is through model compression techniques like pruning and quantization. Quantization is a necessary step in FPGA deployment as it reduces model precision, which in turn reduces memory usage and computational overhead. Channel pruning effectively reduces the model width, reducing computation, but it does not affect the model's depth. This limitation is significant since FPGAs process models sequentially, layer by layer, and the model's depth directly influences the latency

^{*}Equal contribution

[†]Supervising authors.

of execution.

Recognizing these limitations, this paper aims to address the need for customizable CNN models based on specific performance criteria, such as accuracy and latency. By combining techniques like quantization, pruning, and neural architecture search, we seek to identify efficient architectures for FPGAs. Our approach enables the selection of models that prioritize higher accuracy, greater arithmetic intensity, and lower latency. We introduce a new search constraint, the Compute Latency Index (CLI), which integrates latency and arithmetic intensity. Arithmetic intensity encourages the search process to leverage the FPGA’s circuit-level hardware for fast computation while minimizing data access. The weightage parameters defined in the search process guide model selection to meet the required objectives.

Finally, we evaluate the optimized models on the ImageNet dataset. Our results demonstrate that models tailored for specific Xilinx DPU boards on particular FPGA boards can outperform existing models in all evaluated aspects.

II. LITERATURE REVIEW

Zoph et al. [1] introduced a general framework for Neural Architecture Search with RNN controller and reinforcement learning to generate CNN architectures. Since then different variants of this process have been introduced primarily focusing on search strategies. Some of them are Evolutionary Search, Bayesian Optimization [2], ENAS [3], ProxylessNAS [4], DARTS [5], and many others. Later Multi-objective NAS methods were developed to optimize not only the accuracy of the models but also the size or number of parameters.

The Once-for-All (OFA) [6] approach presents a novel perspective in NAS, where a single supernet can be trained to support diverse architectural settings, allowing for efficient deployment across a variety of hardware platforms without the need for retraining. The supernet is progressively made smaller and trained to consist of trained architectures of varying kernel sizes, depth, and width. This method significantly reduces the computational cost and time associated with NAS.

HURRICANE, A Fast Hardware-Aware NAS Framework [7] proposed a larger search space and a two-stage search algorithm, which efficiently generates tailored models for different types of hardware, achieving high accuracy and low latency on various platforms.

HAO, a novel hardware-aware neural architecture search algorithm [8] incorporates integer programming into the search algorithm to prune the design space. Given a set of hardware resource constraints, their integer programming formulation directly outputs the optimal accelerator configuration for mapping a DNN subgraph that minimizes latency

Jiang et al. in [9] proposed a novel reward function in terms of accuracy, latency, and required latency. It then divides convolutional operations into smaller tasks through channel, row, and column tiling and generates a dependency graph between them. Based on the graph, the scheduler determines the sequence of functions for maximizing parallelizability, data reuse, and minimizing pipeline stall.

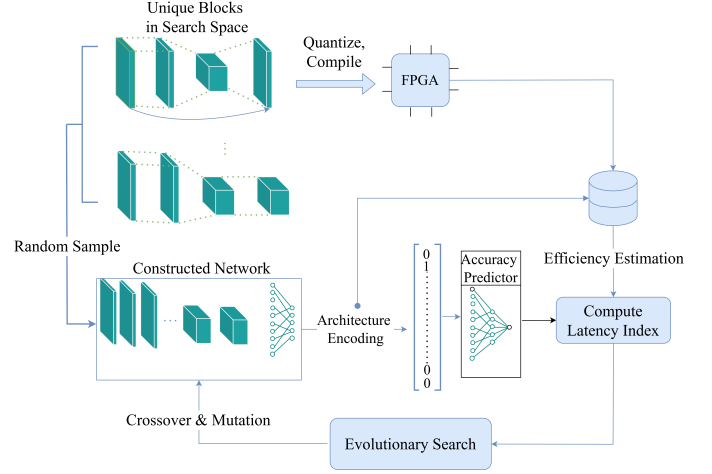


Fig. 1. Our Approach to Architecture Exploration through Evolutionary Neural Architectural Search

Co-design of hardware and CNN search space is an emerging field that aims to optimize both the neural network model and its hardware accelerator. Abdelfattah et al. introduced Codesign-NAS [10], which includes parameters from both the CNN model and the hardware accelerator in the NAS process, jointly searching for the best model-accelerator pair.

Fan et al. [11] presented a three-phase co-design framework that decouples DNN training from the design space exploration of both hardware and neural architectures. Initially, the Hardware design is specified and super-network is trained. Then Gaussian process is used to allow fast estimation of latency, energy, and loss. Finally, genetic algorithm is used to sample networks that perform well in the hardware space.

While previous research has explored deploying CNNs on FPGAs using techniques such as pruning and quantization to achieve real-time performance, there has been limited experimentation in guiding NAS with either latency, arithmetic intensity, or a combined metric tailored for FPGA deployment. Most existing NAS approaches focus on general hardware constraints rather than optimizing for a specific FPGA configuration. Our work aims to focus particularly on FPGA and thus focuses on a single target fixed configuration where the goal is to find the best architecture in terms of accuracy and hardware efficiency for one single target hardware. Still, the same workflow can be easily integrated into several devices.

III. METHODOLOGY

A. Problem Formulation

This study addresses a multi-objective optimization problem of balancing the model’s accuracy and inference latency. Using an FPGA hardware accelerator, we search for CNN architectures with effective tradeoffs between accuracy, performance, and latency. We aim to maximize computation per data access from the MPSoC’s processing system. This is measured by arithmetic intensity (AI), defined as the ratio of multiply-accumulate operations to data size in bytes. We also seek lower latency. To combine these objectives, we introduce the Compute Latency Index (CLI)-the ratio of arithmetic intensity to

latency. A higher CLI indicates better computational efficiency of the network.

We formally define the objective function of our search process as:

$$\arg \max_{a \in A} \left[w_1 \cdot acc(a) + w_2 \cdot \frac{AI(a)}{lat(a, fpga)} \right] \quad (1)$$

The weights w_1 and w_2 control the relative importance of accuracy and computational efficiency.

Arithmetic intensity (AI) is then defined as:

$$AI = \frac{MACs}{Size \text{ in Bytes}} \quad (2)$$

Where MAC is the total number of Multiply accumulate operations.

To constrain our architectures within a specific latency threshold T , we can set the objective for our search algorithm as:

$$\arg \max_{a \in A_T} \left(w_1 \cdot acc(a) + w_2 \cdot \frac{AI(a)}{lat(a, fpga)} \right) \quad (3)$$

Where A_T can be defined as:

$$A_T = \{a \in A \mid lat(a, fpga) \leq T\} \quad (4)$$

We sample $N = 100,000$ architectures from our search space A . For each sampled architecture a_i , we estimate its $acc(a_i)$, $lat(a_i)$, and $AI(a_i)$. We then standardize the acc and CLI incorporated in Eq.1 and Eq.3 using the mean and standard deviation calculated on N . This standardization enables precise tuning of weight values w_1 and w_2 to assign appropriate importance to each term in our objective function.

B. FPGA and DPU specification

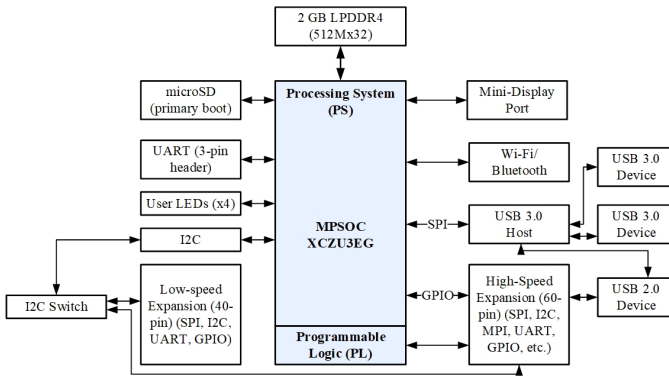


Fig. 2. Functional Block Diagram of MPSoC Highlighting the Communication between Different Elements

The Ultra96-V2 board features a Xilinx Ultrascale+ MPSoC device(XCZU3EG-1SBVA484). The system can be divided into two major components: The Processing System(PS) and the Programming Logic (PL) as shown in Fig 2. The PS contains the ARM-based CPU and DDR RAM. It is responsible for the overall operations. The PL contains the FPGA fabric. The PS and PL are connected through an AXI bus.

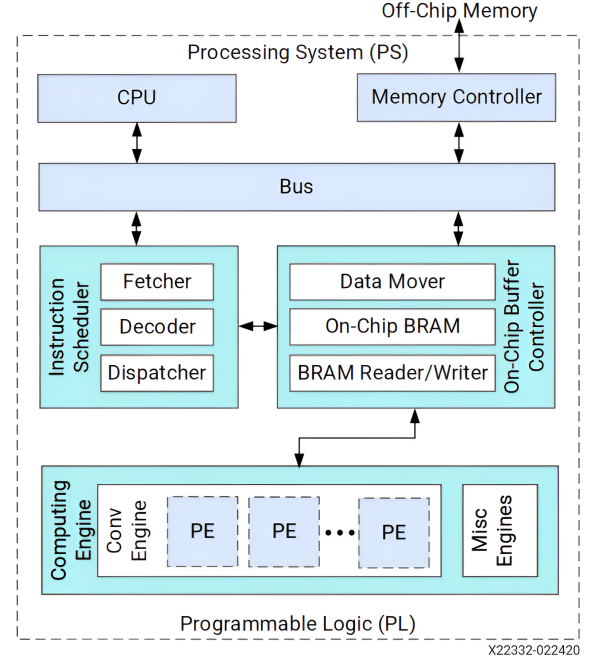


Fig. 3. Abstract Overview of Hardware Architecture of Xilinx's DPU [12]

We used the DPUCZDX8G, a Deep Learning Processing Unit (DPU) designed for MPSoC architectures. The DPU is specifically optimized for CNNs and it operates within the PL. It supports a variety of deep learning models, including VGG, ResNet, and YOLO. Figure 3 provides an abstract representation of the DPUCZDX8G architecture which includes an instruction scheduler, an on-chip buffer controller, and a large set of processing elements called a Computing engine with parallel processing capabilities. 3.

We used the B2304 architecture of the Xilinx DPU, which performs 2304 Multiply-Accumulate (MAC) operations per DPU clock cycle. The design was set to run at 200Mhz with Low RAM consumption. The resource utilization by the design and the overall available resources in the FPGA are presented in the table I.

TABLE I
RESOURCE UTILIZATION IN ULTRA96-V2 (XCZU3EG)

Resource	Utilization	Available	Utilization(%)
LUT	56219	70560	79.68
LUTRAM	4487	28800	15.58
FF	90704	141120	64.27
BRAM	171	216	79.17
DSP	326	369	90.56
BUFG	5	196	2.55

C. Conversion from FP32 to Int8:

Typically, model weights are stored with 32-bit floating-point precision, which delivers high accuracy. However, in resource-constrained environments, quantizing weights to a less precise format is necessary to optimize performance. We quantized and compiled the model using the Vitis AI quantizer. It

converts all model weights and activations to an INT8 fixed-point format.

Approximately 1000 images were randomly sampled from the training set to estimate the range of activation for Post-training static Quantization(PTQ). During inference, the resulting activations are then quantized in this range. While this approach typically results in a 2-5% accuracy loss, this can be mitigated by using Quantization Aware Training (QAT). Both PTQ and QAT are supported by Vitis-AI. The XIR-based compiler maps the model to the DPU instruction set and data flow. While the DPU natively supports a limited number of operations, unsupported operations can be offloaded to the CPU.

We used PetaLinux build and the Vitis AI framework to implement this system. The source code running on the PS uses the same 8-bit fixed-point data type used on the FPGA. For communication with the DPU core, we wrote a host program in Python using the Vitis AI Runtime(VART) API.

D. Search Space

Our search space is based on the MobileNetV3, created for extremely high throughput on mobile and edge devices. The search space contains inverted residual blocks with different configurations. These blocks use Depthwise separable convolutions, and the blocks toward the end of the network use squeeze and excite in the residual layer. We modified the search space and replaced operations that Xilinx's DPU doesn't support with their equivalents, such as Swish and Sigmoid non-linearities with their hard approximations.

- Sigmoid $\Rightarrow$ Hard Sigmoid
- Swish $\Rightarrow$ Hard Swish

Each model sampled from the search space can have 20 or fewer blocks, arranged into 5 stages. Each stage can contain up to 4 blocks and at least a single block as specified by the depth parameter and starts with a stride of 2 to reduce the resolution of the feature map. Overall, the possible arrangements in each block are:

- Kernel sizes: [3, 5, 7]
- Expand ratios: [3, 4, 6]
- Depths: [2, 3, 4]
- Squeeze-Excite Layer: [True, False]

E. Datasets

Our Search space contains a total of 97 unique blocks. We deployed each block on the Ultra96-V2 MPSoC board and calculated its latency for inference to build the latency estimator following the standard Vivado flow. Then, we used the Vitis-AI profiler to measure the latency of individual blocks on the board. To ensure accurate measurements, each block was run 500 times with a batch size of 1, and the average latency was recorded. The raw latency measurements were compiled to build the final dataset that contains the latency of individual blocks along with GOPs, and memory bandwidth. This doesn't include the latency of the GlobalAveragePool block as it is common to all architectures. The dataset is publicly available on GitHub at here.

Algorithm 1 Evolution Algorithm for Neural Architecture Search

```

population  $\leftarrow \emptyset$ 
while  $|population| < P$  do
    model.arch  $\leftarrow$  RANDOM ARCHITECTURE()
    model.score  $\leftarrow F(\text{EP}(\text{model.arch}), \text{AP}(\text{model.arch}))$ 
    add model to population
end while
for iteration  $\leftarrow 1$  to MAX_ITERATIONS do
    sample  $\leftarrow \emptyset$ 
    while  $|sample| < S$  do
        candidate  $\leftarrow$  random architecture from population
        add candidate to sample
    end while
    parents  $\leftarrow$  highest-score models in sample
    child.arch  $\leftarrow$  MUTATE(CROSSOVER(parent.arch))
    child.score  $\leftarrow F(\text{EP}(\text{child.arch}), \text{AP}(\text{child.arch}))$ 
    add child to population
    if  $|population| > P$  then
        remove lowest-score model from population
    end if
end for
return the highest-score model in population

```

F. Accuracy and Efficiency Predictors

Cai et al. [6] have prepared a comprehensive dataset of nearly 60k [architecture, accuracy] pairs which we utilized to build a multi-layer perceptron and train it to build an accuracy predictor that takes a specific architecture encoding as input and predict the estimated accuracy.

$$Latency_{\text{total}} = \sum_{l \in \mathcal{L}} Latency_l \quad (5)$$

$$Arithmetic Intensity_{\text{total}} = \frac{\sum_{l \in \mathcal{L}} MACs_l}{\sum_{l \in \mathcal{L}} Memory_l} \quad (6)$$

To predict the network's efficiency, we used a latency lookup table comprising latency, MACS, and memory specifications for each block in the search space. This works since most FPGAs including Xilinx DPU run the network layer by layer. We pass the encoding, parse it layer by layer, and calculate the total latency following Eq 5 and arithmetic intensity following Eq 6.

IV. RESULTS AND EXPERIMENTS

Our exploration aimed at finding computationally efficient architectures within FPGA environments by minimizing data movement and maximizing computation to address the Von Neumann bottleneck. We evaluated optimization strategies like arithmetic intensity, latency, and CLI. The best trade-off between computational efficiency and model accuracy was achieved using a weighted sum of CLI and accuracy, controlled by weight parameters w_1 and w_2 .

AP Accuracy Predictor
EP Efficiency Predictor

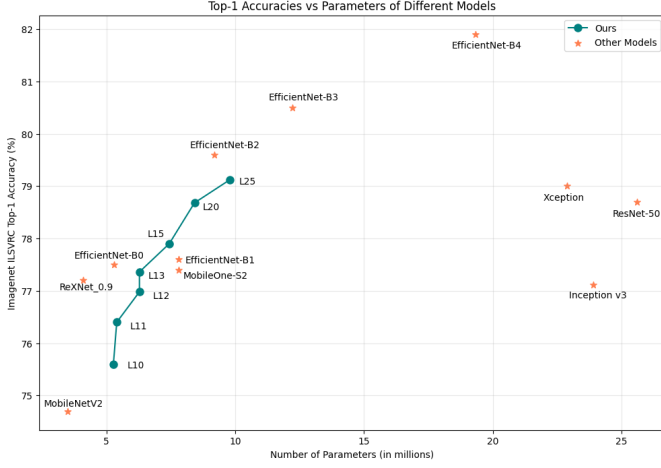


Fig. 4. Comparison of Latency Search Models with Existing Work in terms of Accuracy and Number of Parameters.

We initially explored architectures with high arithmetic intensity, starting with a population of 2500 and running over 1200 generations. The resulting architectures consistently have maximum model depth when the setup is repeated multiple times. For MobileNet-based architectures, deeper networks had greater computations per number of weights but such architectures had high latency on the target FPGA hardware. To address this latency issue, we revised our search constraint by incorporating the latency table we built for the Ultra96v2. A subsequent search was conducted with a population of 5000 architectures for 1200 generations, exploring nearly 4×10^7 architectures. Seven optimal architectures were sampled across different latency requirements ranging from 10 ms to 25 ms. These models outperformed existing networks in terms of parameter efficiency and latency, as shown in Figure 4 and Figure 5. Despite significant improvements in latency, the focus on shallower networks led to reduced computational capacity and smaller kernel sizes, limiting their overall performance.

To balance the conflicting objectives of high arithmetic intensity and low latency, we revised the search objective by adding weights to CLI and accuracy as mentioned in Equation 3 to guide the search process. With varying values of w_1 and w_2 , the population of 5000 architectures evolved for 1800 generations with a mutation and crossover rate of 40% and 30% discovered networks as outlined in the table II. All the architectures were initialized with weights from the OFA Supernet. Further training was conducted on the ILSVRC2012 dataset using stochastic gradient descent (SGD) with a learning rate of 10^{-5} and a momentum of 0.9 for 5 epochs. Validation showed that architectures discovered using the CLI achieved superior performance, achieving a notable balance between computation and latency.

V. DISCUSSION

We observed that maximizing arithmetic intensity increased not only the accuracy but also the network depth and latency. Focusing solely on latency produced shallower networks, sacrificing computational capacity. The CLI approach achieved

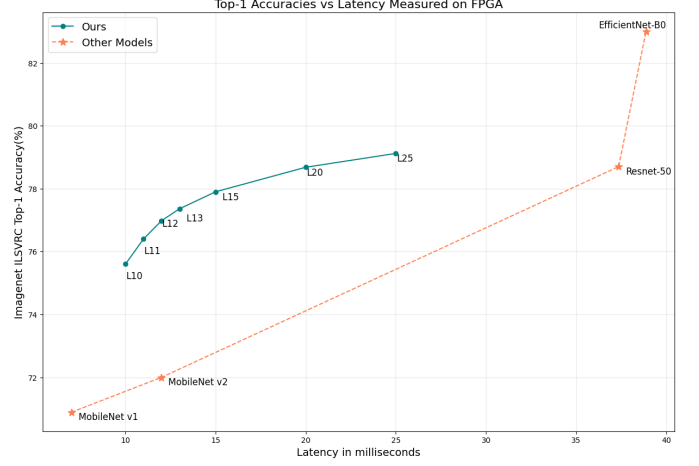


Fig. 5. Latency Comparison of our Searched Models (Latency Constrained) with Existing Work on Ultra96V2

TABLE II
PERFORMANCE OF MODELS SEARCHED WITH WEIGHTED IMPORTANCE

w_1	w_2	<i>Pred</i> Acc(%)	<i>Latency</i> (ms)	<i>CLI</i>	<i>AI</i> (ops/byte)
0.0	1.0	79.23	10.71	1.872	20.05
0.05	0.95	79.39	10.72	1.871	20.06
0.1	0.9	80.1	10.95	1.861	20.38
0.15	0.85	80.1	10.95	1.861	20.38
0.2	0.8	80.47	11.96	1.845	22.07
0.25	0.75	80.8	12.57	1.829	23.0
0.3	0.7	80.8	12.57	1.829	23.0
0.35	0.65	81.18	13.62	1.798	24.5
0.4	0.6	81.38	13.55	1.778	24.1
0.45	0.55	81.51	14.09	1.761	24.82
0.5	0.5	82.19	16.36	1.654	27.07
0.55	0.45	83.07	17.96	1.504	27.02
0.6	0.4	84.48	21.54	1.203	25.93
0.65	0.35	84.97	23.14	1.091	25.26
0.7	0.3	85.31	27.35	0.929	25.42
0.75	0.25	85.31	27.35	0.929	25.42
0.8	0.2	85.4	28.14	0.869	24.47
0.85	0.15	85.4	28.14	0.869	24.47
0.9	0.1	85.09	29.77	0.846	25.21
0.95	0.05	85.15	30.48	0.854	26.04
1.0	0.0	85.15	30.48	0.854	26.04

the best balance between computational efficiency and latency. Our models used fewer parameters, had lower inference times on FPGA, and attained higher accuracy compared to existing networks.

To validate our findings, we deployed several searched architectures using the standard quantization-compilation workflow for AMD FPGAs and measured their inference speed. Figure 6 illustrates a consistent discrepancy between estimated and measured values in proportion to network size. This discrepancy can largely be attributed to the inherent overhead of FPGA configurations, routing, and interconnection. Additionally, we identified that inter-block communication introduces further delays, emphasizing the need for efficient data transfer

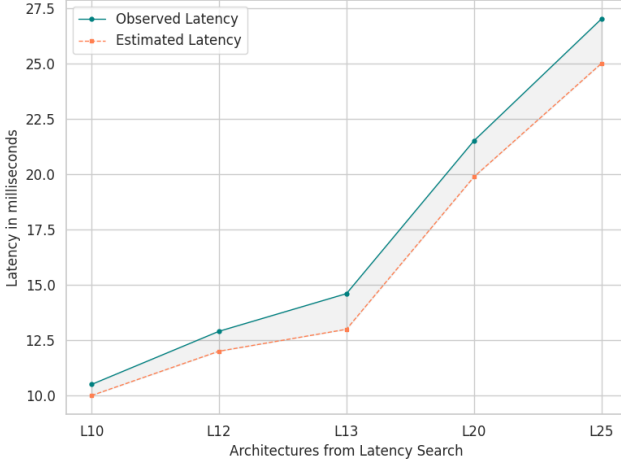


Fig. 6. Comparison of Estimated Latency of Our Predictor with Actual Latency Measured on Ultra96V2

and synchronization strategies to optimize performance.

VI. CONCLUSION

Given the challenges of deploying real-time applications on edge devices, our approach emphasizes the need to balance efficiency and accuracy. This trade-off is crucial, as real-world applications impose constraints that go beyond accuracy alone. To address this, we experimented with different search strategies constrained by arithmetic intensity, latency, and a weighted importance criterion to evaluate their effectiveness. We then developed a CLI-based workflow to efficiently select an architecture using two tunable parameters, w_1 and w_2 , which account for latency and performance requirements. This enables the selection of networks that could have better performance, efficiency, and optimization for a specific FPGA target device.

By integrating new objective functions into existing neural architecture search (NAS) methods, we adapted traditional NAS methodologies and introduced CLI, a novel metric that optimally balances accuracy, latency, and computational efficiency. A similar approach of creating a specialized latency dataset for the Ultra96V2 based on MobileNetV3 can be extended to derive latency data for other FPGA boards, allowing integration into our NAS process to discover architectures for specific FPGA hardware. Unlike conventional NAS explorations, which often require several days, our search leverages two heuristic estimators: accuracy and latency to possibly achieve better architectures. Additionally, weight transfer from OFA accelerates the ImageNet training process, completing the search within a few GPU hours. While all our current experiments consider latency and computation capacity targeting image classification tasks, a similar workflow can be followed to include constraints such as power consumption, model size, and underlying hardware architecture with border applicability in object detection and image segmentation tasks.

REFERENCES

[1] B. Zoph and Q. V. Le, “Neural architecture search with reinforcement learning,” *arXiv preprint arXiv:1611.01578*, 2016.

[2] C. White, W. Neiswanger, and Y. Savani, “Bananas: Bayesian optimization with neural architectures for neural architecture search,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 10 293–10 301.

[3] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, “Efficient neural architecture search via parameters sharing,” in *International conference on machine learning*. PMLR, 2018, pp. 4095–4104.

[4] H. Cai, L. Zhu, and S. Han, “Proxylessnas: Direct neural architecture search on target task and hardware,” *arXiv preprint arXiv:1812.00332*, 2018.

[5] H. Liu, K. Simonyan, and Y. Yang, “Darts: Differentiable architecture search,” *arXiv preprint arXiv:1806.09055*, 2018.

[6] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, “Once-for-all: Train one network and specialize it for efficient deployment,” *arXiv preprint arXiv:1908.09791*, 2019.

[7] L. L. Zhang, Y. Yang, Y. Jiang, W. Zhu, and Y. Liu, “Fast hardware-aware neural architecture search,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020, pp. 692–693.

[8] Z. Dong, Y. Gao, Q. Huang, J. Wawrzyniec, H. K. So, and K. Keutzer, “Hao: Hardware-aware neural architecture optimization for efficient inference,” in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2021, pp. 50–59.

[9] W. Jiang, X. Zhang, E. H.-M. Sha, L. Yang, Q. Zhuge, Y. Shi, and J. Hu, “Accuracy vs. efficiency: Achieving both through fpga-implementation aware neural architecture search,” in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[10] M. S. Abdelfattah, Ł. Dudziak, T. Chau, R. Lee, H. Kim, and N. D. Lane, “Best of both worlds: Automl codesign of a cnn and its hardware accelerator,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.

[11] H. Fan, M. Ferianc, Z. Que, H. Li, S. Liu, X. Niu, and W. Luk, “Algorithm and hardware co-design for reconfigurable cnn accelerator,” in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 250–255.

[12] AMD, *DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)*, 2024, accessed: 2024-01-27. [Online]. Available: <https://docs.amd.com/r/en-US/pg338-dpu>